B2B Travel Booking & Revenue Analytics

PHASE 1 - RAW DATA PROFILING & DATA QUALITY

Complete Beginner-Friendly Documentation

What to do | How to do it | Why to do it | Concepts | SQL | Results | Analyst thinking

Purpose of this guide

This document captures the complete Phase 1 learning journey in a classroom-style format. It explains each profiling task before cleaning begins, using the exact PostgreSQL concepts and the actual results found in the project dataset.

Project stack: PostgreSQL / pgAdmin | SQL | Python | Power BI | Excel

Phase 1 status: COMPLETE

1. Phase 1 Overview

Phase 1 is the raw-data profiling and data-quality assessment stage. The goal is not to clean immediately. The goal is to understand what is in the data, identify problems, measure their size, and document what should be handled later in the staging and clean layers.

Core analyst mindset

Detect -> Investigate -> Understand -> Document -> Decide. Do not delete unusual records just because they look wrong.

What Phase 1 answers

- How many rows and unique business records are present?
- Are primary-key-like IDs duplicated?
- Which important fields are missing?
- Are dates stored consistently and logically?
- Do numeric values make business sense?
- Are categorical labels consistent?
- Do foreign keys match their dimension tables?
- Are bookings, cancellations, and payments logically consistent?
- Could one-to-many joins cause double counting?

Project tables used in Phase 1

Table	Expected grain	Main role in Phase 1
raw.bookings	One row per booking	Main fact table; duplicates, dates, amounts, status, foreign keys
raw.travel_agents	One row per agent	Agent reference checks and country-name consistency
raw.hotels	One row per hotel	Hotel reference integrity
raw.suppliers	One row per supplier	Supplier reference integrity
raw.destinations	One row per destination	Destination reference integrity
raw.cancellations	One row per cancellation event	Cancellation consistency and orphan records
raw.payments	One row per payment transaction	One-to-many behavior and double-counting risk

Contents

- Lessons 1-7: basic inspection, row counts, duplicates, and missing values
- Lessons 8-11: date formats, date conversion, invalid date sequence, and lead time
- Lessons 12-17: numeric quality and business-rule checks
- Lessons 18-22: categorical profiling and text standardization concepts
- Lessons 23-29: foreign-key and cancellation consistency checks
- Lessons 30-33: payment grain, double counting, cancelled+paid, and refund inspection
- Final issue log, decision framework, common mistakes, and reusable SQL appendix

2. Complete Lessons

Lesson 1: Inspect the raw bookings table

Concept: Basic data inspection and table grain

WHAT TO DO	HOW TO DO IT	WHY IT MATTERS
Look at a small sample of raw booking rows before writing complex analysis.	Use SELECT * with LIMIT so you can see the columns and values without returning the whole table.	You need a mental picture of the table: what one row means, which columns are important, and whether values look reasonable.

SQL

```
SELECT *
FROM raw.bookings
LIMIT 10;
```

How the SQL works

SELECT means "show data".

* means "all columns".

FROM raw.bookings identifies the source table.

LIMIT 10 returns only ten rows, making inspection easy.

Your actual result

First booking inspected: B0000001. Status: Completed. Gross booking value: 9522.99.

Main takeaway

Before profiling a table, understand its grain. In this project, one raw booking row is intended to represent one booking record.

Lesson 2: Count the booking rows

Concept: COUNT(*) and dataset size

WHAT TO DO	HOW TO DO IT	WHY IT MATTERS
Measure how many records are in the raw bookings table.	Use COUNT(*) on the table.	Row count is the baseline for later checks. After cleaning, you can compare raw vs clean row counts and explain any difference.

SQL

```
SELECT COUNT(*)
FROM raw.bookings;
```

How the SQL works

COUNT(*) counts every row returned by the query.

Your actual result

Total raw booking rows: 120,175.

Main takeaway
Always establish a row-count baseline before cleaning or joining tables.

Lesson 3: Compare total rows with unique booking IDs

Concept: DISTINCT and uniqueness

WHAT TO DO	HOW TO DO IT	WHY IT MATTERS
Check whether every booking row has a unique booking_id.	Compare COUNT(*) with COUNT(DISTINCT booking_id), then subtract the two.	If total rows exceed unique IDs, some booking IDs repeat. Duplicates can overstate bookings, GBV, revenue, and other KPIs.

SQL

```
SELECT
    COUNT(*) AS total_rows,
    COUNT(DISTINCT booking_id) AS unique_booking_ids,
    COUNT(*) - COUNT(DISTINCT booking_id) AS duplicate_rows
FROM raw.bookings;
```

How the SQL works

DISTINCT removes repeated values before counting.

The difference between total rows and unique IDs shows extra repeated rows.

Your actual result
total_rows = 120,175; unique_booking_ids = 120,000; duplicate_rows = 175.

Main takeaway
A uniqueness check is one of the first tests for any column that behaves like a primary key.

Lesson 4: Find which booking IDs are duplicated

Concept: GROUP BY and HAVING

WHAT TO DO	HOW TO DO IT	WHY IT MATTERS
Identify the specific booking IDs that appear more than once.	Group by booking_id and keep only groups with COUNT(*) > 1.	Knowing there are duplicates is not enough. You also need to know which IDs are affected so you can inspect them.

SQL

```
SELECT
    booking_id,
    COUNT(*) AS row_count
FROM raw.bookings
GROUP BY booking_id
HAVING COUNT(*) > 1
ORDER BY row_count DESC, booking_id;
```

How the SQL works

GROUP BY puts rows with the same booking_id together.

HAVING filters groups after aggregation. WHERE filters individual rows before aggregation.

Your actual result

175 duplicate booking IDs were shown. First observed duplicate: B0000944 with row_count = 2.

Main takeaway

Use HAVING when the condition depends on an aggregate such as COUNT, SUM, or AVG.

Lesson 5: Inspect a duplicate in full

Concept: Exact duplicate vs conflicting duplicate

WHAT TO DO	HOW TO DO IT	WHY IT MATTERS
Compare all columns for one duplicated booking ID.	Filter the raw table to the chosen booking ID and visually compare both rows.	An exact duplicate can usually be deduplicated later. A conflicting duplicate needs investigation because the rows disagree.

SQL

```
SELECT *
FROM raw.bookings
WHERE booking_id = 'B0000944';
```

How the SQL works

WHERE keeps only rows matching the condition.

Your actual result

Both B0000944 rows were exactly the same.

Main takeaway

Do not delete from the raw layer. Preserve raw data for traceability; deduplicate in staging/clean layers later.

Lesson 6: Check missing supplier IDs

Concept: NULL, blank strings, and TRIM()

WHAT TO DO	HOW TO DO IT	WHY IT MATTERS
Count bookings where supplier_id is missing.	Check both IS NULL and an empty string after TRIM().	Missing values can appear as database NULLs, empty strings, or strings containing only spaces.

SQL

```
SELECT
    COUNT(*) AS missing_supplier_ids
FROM raw.bookings
WHERE supplier_id IS NULL
   OR TRIM(supplier_id) = '';
```

How the SQL works

IS NULL checks a true SQL NULL.

TRIM(supplier_id) removes leading/trailing spaces. If the result equals '', the value is blank.

Your actual result
Missing supplier IDs: 240.

Main takeaway
A missing supplier does not automatically make the whole booking unusable. The booking may still support agent, hotel, destination, booking-volume, and some revenue analysis.

Be careful
Missing data should be evaluated by use case, not automatically deleted.

Lesson 7: Profile multiple missing fields in one query

Concept: PostgreSQL FILTER on aggregates

WHAT TO DO	HOW TO DO IT	WHY IT MATTERS
Count missing values for several important columns at the same time.	Use COUNT(*) FILTER (WHERE condition) for each field.	FILTER lets one SELECT calculate many conditional counts efficiently and clearly.

SQL

```
SELECT
    COUNT(*) FILTER (WHERE booking_id IS NULL OR TRIM(booking_id) = '') AS missing_booking_id,
    COUNT(*) FILTER (WHERE booking_date IS NULL OR TRIM(booking_date) = '') AS missing_booking_date,
    COUNT(*) FILTER (WHERE travel_date IS NULL OR TRIM(travel_date) = '') AS missing_travel_date,
    COUNT(*) FILTER (WHERE agent_id IS NULL OR TRIM(agent_id) = '') AS missing_agent_id,
    COUNT(*) FILTER (WHERE hotel_id IS NULL OR TRIM(hotel_id) = '') AS missing_hotel_id,
    COUNT(*) FILTER (WHERE supplier_id IS NULL OR TRIM(supplier_id) = '') AS missing_supplier_id,
    COUNT(*) FILTER (WHERE destination_id IS NULL OR TRIM(destination_id) = '') AS
missing_destination_id,
    COUNT(*) FILTER (WHERE gross_booking_value IS NULL OR TRIM(gross_booking_value) = '') AS
missing_gbv
FROM raw.bookings;
```

How the SQL works

COUNT(*) counts rows.

FILTER tells COUNT which rows to use.

Example: COUNT(*) FILTER (WHERE booking_status = 'Cancelled') counts only cancelled rows.

Your actual result
For the key fields reviewed, booking_id, agent_id, hotel_id, destination_id, and GBV had 0 missing; supplier_id had 240 missing.

Main takeaway
FILTER is a powerful PostgreSQL feature for KPI and data-quality profiling.

Lesson 8: Detect booking-date formats

Concept: Regular expressions and CASE classification

WHAT TO DO	HOW TO DO IT	WHY IT MATTERS
Measure how many booking_date values follow each known text format.	Use a CASE expression with regular-expression patterns for YYYY-MM-DD and DD/MM/YYYY.	Dates stored as text may use multiple formats. Converting them safely requires knowing what formats exist.

SQL

```
SELECT
    CASE
        WHEN booking_date ~ '^\d{4}-\d{2}-\d{2}$' THEN 'YYYY-MM-DD'
        WHEN booking_date ~ '^\d{2}/\d{2}/\d{4}$' THEN 'DD/MM/YYYY'
        WHEN booking_date IS NULL OR TRIM(booking_date) = '' THEN 'NULL/BLANK'
        ELSE 'OTHER/INVALID'
    END AS date_format,
    COUNT(*) AS row_count
FROM raw.bookings
GROUP BY 1
ORDER BY row_count DESC;
```

How the SQL works

The ~ operator checks whether text matches a regular expression.

^\d{4}-\d{2}-\d{2}$ means four digits, dash, two digits, dash, two digits.

^\d{2}/\d{2}/\d{4}$ means two digits, slash, two digits, slash, four digits.

GROUP BY 1 groups by the first selected expression: date_format.

Your actual result

YYYY-MM-DD = 120,014; DD/MM/YYYY = 161; NULL/BLANK = 0; OTHER/INVALID = 0.

Main takeaway

The dates were valid-looking but inconsistent in formatting. This is a standardization issue, not necessarily a correctness issue.

Lesson 9: Convert text dates safely

Concept: Type casting and TO_DATE()

WHAT TO DO	HOW TO DO IT	WHY IT MATTERS
Convert both known booking-date formats into a real PostgreSQL DATE value.	Use CASE to select the correct conversion method for each format.	Date arithmetic and reliable filtering should be performed on DATE values, not mixed-format text.

SQL

```
SELECT
    booking_id,
    booking_date,
    CASE
        WHEN booking_date ~ '^\d{4}-\d{2}-\d{2}$' THEN booking_date::date
        WHEN booking_date ~ '^\d{2}/\d{2}/\d{4}$' THEN TO_DATE(booking_date, 'DD/MM/YYYY')
        ELSE NULL
    END AS clean_booking_date
FROM raw.bookings
LIMIT 20;
```

How the SQL works

::date casts a compatible text value into the DATE data type.

TO_DATE(text, 'DD/MM/YYYY') tells PostgreSQL exactly how to interpret that text.

Your actual result

The dates converted successfully and displayed consistently as YYYY-MM-DD.

Main takeaway

Use safe conditional parsing when raw text can arrive in more than one date format.

Lesson 10: Find travel dates before booking dates

Concept: CTEs and date-sequence business rules

WHAT TO DO	HOW TO DO IT	WHY IT MATTERS
Identify bookings where travel_dt < booking_dt.	Parse both date fields in a CTE, then filter the parsed result.	A normal booking should be created on or before the travel date. A negative sequence can indicate data-entry or source-system problems.

SQL

```
WITH parsed_dates AS (
    SELECT
        booking_id,
        CASE
            WHEN booking_date ~ '^\d{4}-\d{2}-\d{2}$' THEN booking_date::date
            WHEN booking_date ~ '^\d{2}/\d{2}/\d{4}$' THEN TO_DATE(booking_date, 'DD/MM/YYYY')
            ELSE NULL
        END AS booking_dt,
        CASE
            WHEN travel_date ~ '^\d{4}-\d{2}-\d{2}$' THEN travel_date::date
            WHEN travel_date ~ '^\d{2}/\d{2}/\d{4}$' THEN TO_DATE(travel_date, 'DD/MM/YYYY')
            ELSE NULL
        END AS travel_dt
    FROM raw.bookings
)
SELECT booking_id, booking_dt, travel_dt
FROM parsed_dates
WHERE travel_dt < booking_dt;
```

How the SQL works

WITH parsed_dates AS (...) creates a named temporary result for the query.

The outer query uses the parsed DATE columns.

travel_dt < booking_dt is the business-rule check.

Your actual result
25 rows failed the sequence check. First observed booking: B0000498.

Main takeaway
CTEs make multi-step SQL easier to read by separating preparation from analysis.

Lesson 11: Calculate lead time

Concept: Date arithmetic and derived metrics

WHAT TO DO	HOW TO DO IT	WHY IT MATTERS
Calculate how many days occur between booking and travel.	Subtract booking_dt from travel_dt after converting both to DATE.	Lead time is a useful travel-industry measure for planning behavior, last-minute booking patterns, and later cancellation analysis.

SQL

```
WITH parsed_dates AS (
    SELECT
        booking_id,
        CASE
            WHEN booking_date ~ '^\d{4}-\d{2}-\d{2}$' THEN booking_date::date
            WHEN booking_date ~ '^\d{2}/\d{2}/\d{4}$' THEN TO_DATE(booking_date, 'DD/MM/YYYY')
            ELSE NULL
        END AS booking_dt,
        CASE
            WHEN travel_date ~ '^\d{4}-\d{2}-\d{2}$' THEN travel_date::date
            WHEN travel_date ~ '^\d{2}/\d{2}/\d{4}$' THEN TO_DATE(travel_date, 'DD/MM/YYYY')
            ELSE NULL
        END AS travel_dt
    FROM raw.bookings
)
SELECT
    booking_id,
    booking_dt,
    travel_dt,
    travel_dt - booking_dt AS lead_days
FROM parsed_dates
LIMIT 10;
```

How the SQL works

PostgreSQL DATE - DATE returns the number of days between the dates.

A negative lead_days value corresponds to the invalid sequence identified in Lesson 10.

Your actual result
For B0000001: booking_dt = 2025-05-20; travel_dt = 2025-06-12; lead_days = 23.

Main takeaway
Lead time = travel date - booking date.

Lesson 12: Check rooms less than or equal to zero

Concept: Numeric casting and domain rules

WHAT TO DO	HOW TO DO IT	WHY IT MATTERS
Find bookings where rooms <= 0.	Cast the raw text field to numeric and apply the business rule.	A hotel booking normally requires at least one room. Zero or negative room counts can distort room-night and booking metrics.

SQL

```
SELECT
    booking_id,
    rooms
FROM raw.bookings
WHERE rooms::numeric <= 0;
```

How the SQL works

::numeric converts text-like numeric values into a numeric type so mathematical comparison works.

Your actual result

20 rows had rooms <= 0. The first observed invalid rooms value was 0.

Main takeaway

Numeric validity is not only about data type; it is also about business meaning.

Be careful

In a less controlled dataset, first verify that values are numeric before casting. Text such as "two" would cause a cast error.

Lesson 13: Check nights less than or equal to zero

Concept: Business-rule validation for stay duration

WHAT TO DO	HOW TO DO IT	WHY IT MATTERS
Find bookings with nights <= 0.	Apply the same numeric-casting pattern to nights.	A hotel stay normally requires at least one night. Negative or zero nights can invalidate room-night, rate, and profitability analysis.

SQL

```
SELECT
    booking_id,
    nights
FROM raw.bookings
WHERE nights::numeric <= 0;
```

How the SQL works

The query pattern is reusable: choose a field, cast if needed, then test a domain rule.

Your actual result

20 rows were returned. First observed booking: B0007064 with nights = -1.

Main takeaway
Repeated query patterns are valuable: you can apply one data-quality idea consistently across several numeric fields.

Lesson 14: Check non-positive gross booking value

Concept: Financial domain validation

WHAT TO DO	HOW TO DO IT	WHY IT MATTERS
Find bookings where gross_booking_value <= 0.	Convert GBV to numeric and filter for zero or negative values.	GBV is central to revenue, ABV, margin, and growth metrics. Non-positive values can materially distort KPIs.

SQL

```
SELECT
    booking_id,
    gross_booking_value
FROM raw.bookings
WHERE gross_booking_value::numeric <= 0;
```

How the SQL works

The check detects zero and negative values, not merely NULLs.

Your actual result
58 rows had GBV <= 0. First observed booking: B0002083 with GBV = 0.

Main takeaway
A value can be present but still be invalid for the business context.

Lesson 15: Check supplier cost greater than GBV

Concept: Cross-field business rules

WHAT TO DO	HOW TO DO IT	WHY IT MATTERS
Find bookings where supplier_cost > gross_booking_value.	Compare two numeric fields from the same row.	This can indicate a loss-making booking, pricing issue, promotion, unusual commercial arrangement, or bad data. It deserves investigation.

SQL

```
SELECT
    booking_id,
    gross_booking_value,
    supplier_cost
FROM raw.bookings
WHERE supplier_cost::numeric > gross_booking_value::numeric;
```

How the SQL works

Cross-field rules compare values that should have a logical relationship.

Your actual result
58 rows met the condition. B0002083 had GBV = 0 and supplier_cost = 51,401.42.

Main takeaway
Do not label supplier_cost > GBV as automatically wrong. A loss can be real; context matters.

Lesson 16: Calculate margin before discount

Concept: Calculated columns and aliases

WHAT TO DO	HOW TO DO IT	WHY IT MATTERS
Measure GBV - supplier cost for suspicious rows.	Create an expression and name it with AS margin_before_discount.	A calculated value helps quantify the size of a potential loss or anomaly rather than only flagging it.

SQL

```
SELECT
    booking_id,
    gross_booking_value,
    supplier_cost,
    gross_booking_value::numeric - supplier_cost::numeric AS margin_before_discount
FROM raw.bookings
WHERE supplier_cost::numeric > gross_booking_value::numeric
LIMIT 10;
```

How the SQL works

AS gives a readable name to a calculated expression.

This calculation intentionally excludes discount_amount; it is a simple intermediate diagnostic.

Your actual result
For B0002083, margin_before_discount = -51,401.42.

Main takeaway
Calculated columns make data-quality checks more informative by showing magnitude, not just pass/fail.

Lesson 17: Check discount greater than GBV

Concept: Another cross-field financial rule

WHAT TO DO	HOW TO DO IT	WHY IT MATTERS
Find rows where discount_amount > gross_booking_value.	Compare the discount and GBV after numeric casting.	A discount larger than the booking value is unusual and may signal an adjustment, refund-related process, source issue, or pricing problem.

SQL

```
SELECT
    booking_id,
    gross_booking_value,
    discount_amount
FROM raw.bookings
WHERE discount_amount::numeric > gross_booking_value::numeric;
```

How the SQL works

The same cross-field pattern from Lesson 15 is reused here.

Your actual result

39 rows were returned. B0002083 had GBV = 0 and discount_amount = 578.98.

Main takeaway

One row can fail several checks at the same time. B0002083 is a good example of a record that needs careful investigation.

Lesson 18: Profile booking-status categories

Concept: Categorical frequency distribution

WHAT TO DO	HOW TO DO IT	WHY IT MATTERS
List each distinct booking status and its row count.	GROUP BY booking_status and COUNT(*) each group.	Category profiling reveals spelling, case, blank, and unexpected-label problems before dashboarding.

SQL

```
SELECT
    booking_status,
    COUNT(*) AS row_count
FROM raw.bookings
GROUP BY booking_status
ORDER BY row_count DESC;
```

How the SQL works

ORDER BY row_count DESC shows the most common category first.

Your actual result

Completed = 101,135; Cancelled = 13,995; Confirmed = 5,045. These total 120,175 rows.

Main takeaway

The booking_status labels were consistent in the observed raw data: three clean categories.

Lesson 19: Profile currencies

Concept: Multi-currency awareness

WHAT TO DO	HOW TO DO IT	WHY IT MATTERS
List all currencies used by bookings and count them.	Group by currency and count rows.	You must know the currency mix before financial aggregation. Amounts in different currencies cannot be meaningfully summed without normalization.

SQL

```
SELECT
    currency,
    COUNT(*) AS row_count
FROM raw.bookings
GROUP BY currency
ORDER BY row_count DESC;
```

How the SQL works

This is the same categorical-frequency pattern used for booking status.

Your actual result

15 currencies were present. Counts: EUR 27,964; USD 10,209; SAR 9,249; GBP 8,952; MYR 8,540; EGP 8,401; TRY 8,055; JPY 7,553; AED 5,882; QAR 5,430; IDR 5,356; SGD 4,841; THB 4,513; INR 2,752; AUD 2,478.

Main takeaway

Do not sum raw GBV across currencies. Convert to a common reporting currency first.

Lesson 20: Check country-name consistency

Concept: Categorical standardization problems

WHAT TO DO	HOW TO DO IT	WHY IT MATTERS
Inspect the different country labels in travel_agents.	Group by country and count rows so variants appear as separate categories.	Different spellings, cases, abbreviations, and punctuation split one business category into several report categories.

SQL

```
SELECT
    country,
    COUNT(*) AS row_count
FROM raw.travel_agents
GROUP BY country
ORDER BY row_count DESC;
```

How the SQL works

SQL compares text literally unless you standardize it. India, INDIA, and india are separate raw values.

Your actual result

Observed examples included India / INDIA / india, U.A.E, and Signapore, among other variants.

Main takeaway
Categorical inconsistency can make Power BI show the same country multiple times.

Lesson 21: Use TRIM() and UPPER() to normalize text for comparison

Concept: Text-cleaning functions

WHAT TO DO	HOW TO DO IT	WHY IT MATTERS
See how spacing and capitalization can be standardized without changing the raw table.	Create derived columns using TRIM(country) and UPPER(TRIM(country)).	Normalizing text makes it easier to recognize values that mean the same thing despite case or extra spaces.

SQL

```
SELECT
    country AS original_country,
    TRIM(country) AS trimmed_country,
    UPPER(TRIM(country)) AS cleaned_country
FROM raw.travel_agents
GROUP BY country
ORDER BY country;
```

How the SQL works

TRIM removes leading and trailing spaces.

UPPER converts text to uppercase for consistent comparison.

Your actual result
Examples observed: india / India / INDIA -> INDIA; germany / Germany -> GERMANY; italy / Italy / ITALY -> ITALY.

Main takeaway
TRIM + UPPER solves case/spacing issues, but it cannot automatically fix misspellings such as Signapore or semantic variants such as U.A.E vs United Arab Emirates.

Lesson 22: Standardize known country variants with CASE

Concept: Rule-based categorical cleaning

WHAT TO DO	HOW TO DO IT	WHY IT MATTERS
Map known variants and spelling mistakes to one standard label.	Use CASE, UPPER(TRIM()), IN, and INITCAP().	A consistent dimension value is required for accurate grouping, filtering, mapping, and dashboard labels.

SQL

```
SELECT
    country AS original_country,
    CASE
        WHEN UPPER(TRIM(country)) IN ('UAE', 'U.A.E', 'UNITED ARAB EMIRATES')
            THEN 'United Arab Emirates'
        WHEN UPPER(TRIM(country)) = 'SIGNAPORE'
            THEN 'Singapore'
        ELSE INITCAP(TRIM(country))
    END AS standardized_country
FROM raw.travel_agents
GROUP BY country
ORDER BY country;
```

How the SQL works

IN checks whether the normalized value matches any value in a list.

INITCAP changes ordinary labels to title-style capitalization, such as SAUDI ARABIA -> Saudi Arabia.

Your actual result

Observed standardization worked for India variants, Germany variants, Italy variants, Saudi Arabia variants, and similar labels.

Main takeaway

In Phase 1 this is a test of the cleaning logic. The raw table remains unchanged; the rule belongs in staging/clean layers later.

Lesson 23: Check whether every booking agent exists

Concept: LEFT JOIN and referential integrity

WHAT TO DO	HOW TO DO IT	WHY IT MATTERS
Find booking agent_ids that have no matching row in travel_agents.	LEFT JOIN bookings to travel_agents and keep rows where the right-side agent_id is NULL.	A missing foreign-key match means booking facts cannot be reliably enriched with that agent's attributes.

SQL

```
SELECT
    b.booking_id,
    b.agent_id
FROM raw.bookings AS b
LEFT JOIN raw.travel_agents AS a
    ON b.agent_id = a.agent_id
WHERE a.agent_id IS NULL;
```

How the SQL works

AS b and AS a are short table aliases.

LEFT JOIN keeps every booking and tries to attach a matching agent.

If no match exists, columns from a are NULL.

Your actual result

1,000 booking rows had no matching agent. First observed booking_id: B0000002.

Main takeaway
LEFT JOIN + right-side IS NULL is a standard pattern for finding missing references.

Lesson 24: Check hotel references

Concept: Reusable foreign-key validation

WHAT TO DO	HOW TO DO IT	WHY IT MATTERS
Find booking hotel_ids that do not exist in hotels.	Use the same LEFT JOIN pattern with raw.hotels.	Foreign-key quality should be checked independently for every important relationship.

SQL

```
SELECT
    b.booking_id,
    b.hotel_id
FROM raw.bookings AS b
LEFT JOIN raw.hotels AS h
    ON b.hotel_id = h.hotel_id
WHERE h.hotel_id IS NULL;
```

How the SQL works

Only the joined table and key names change; the integrity-check pattern stays the same.

Your actual result
0 rows were returned. Every booking hotel_id matched a hotel.

Main takeaway
A zero-result quality check is still valuable evidence: it confirms that relationship is healthy.

Lesson 25: Separate missing supplier IDs from invalid supplier references

Concept: Missing values vs orphan foreign keys

WHAT TO DO	HOW TO DO IT	WHY IT MATTERS
Among bookings that actually contain a supplier_id, check whether that ID exists in suppliers.	Exclude NULL/blank supplier IDs first, then test for missing join matches.	A blank supplier_id and a nonblank-but-invalid supplier_id are different data-quality problems and should be documented separately.

SQL

```
SELECT
    b.booking_id,
    b.supplier_id
FROM raw.bookings AS b
LEFT JOIN raw.suppliers AS s
    ON b.supplier_id = s.supplier_id
WHERE b.supplier_id IS NOT NULL
  AND TRIM(b.supplier_id) <> ''
  AND s.supplier_id IS NULL;
```

How the SQL works

<> means 'not equal to'.

The first two WHERE conditions remove the already-known missing supplier cases.

The final condition finds real unmatched references.

Your actual result

Invalid nonblank supplier references = 0. Separately, 240 bookings had missing supplier_id.

Main takeaway

Document each failure type precisely; do not mix missing values with referential-integrity failures.

Lesson 26: Check destination references

Concept: Foreign-key validation for destination

WHAT TO DO	HOW TO DO IT	WHY IT MATTERS
Find booking destination_ids that do not exist in destinations.	LEFT JOIN bookings to destinations and filter unmatched rows.	Destination analysis depends on reliable links from bookings to the destination dimension.

SQL

```
SELECT
    b.booking_id,
    b.destination_id
FROM raw.bookings AS b
LEFT JOIN raw.destinations AS d
    ON b.destination_id = d.destination_id
WHERE d.destination_id IS NULL;
```

How the SQL works

This repeats the same integrity-check template used for agents and hotels.

Your actual result

0 invalid destination references.

Main takeaway

The booking-to-destination relationship passed the profiling check.

Lesson 27: Check cancelled bookings have cancellation records

Concept: Cross-table business consistency

WHAT TO DO	HOW TO DO IT	WHY IT MATTERS
Find bookings marked Cancelled that have no matching cancellation row.	LEFT JOIN cancelled bookings to cancellations and filter where the cancellation match is NULL.	Status in the booking fact should agree with the operational cancellation table.

SQL

```
SELECT DISTINCT
    b.booking_id
FROM raw.bookings AS b
LEFT JOIN raw.cancellations AS c
    ON b.booking_id = c.booking_id
WHERE b.booking_status = 'Cancelled'
  AND c.booking_id IS NULL;
```

How the SQL works

DISTINCT avoids reporting the same booking more than once if raw bookings contains duplicate rows.

Your actual result

0 rows. Every booking marked Cancelled had a matching cancellation record.

Main takeaway

Consistency checks compare business meaning across tables, not just keys inside one table.

Lesson 28: Check cancellation records are linked to cancelled bookings

Concept: Reverse consistency check

WHAT TO DO	HOW TO DO IT	WHY IT MATTERS
Find cancellation rows whose booking exists but is not marked Cancelled.	Join cancellations to bookings, require an existing booking, then filter booking_status <> 'Cancelled'.	Checking both directions catches problems that a one-direction test can miss.

SQL

```
SELECT DISTINCT
    c.booking_id,
    b.booking_status
FROM raw.cancellations AS c
LEFT JOIN raw.bookings AS b
    ON c.booking_id = b.booking_id
WHERE b.booking_id IS NOT NULL
  AND b.booking_status <> 'Cancelled';
```

How the SQL works

b.booking_id IS NOT NULL limits this check to cancellations whose booking really exists.

<> 'Cancelled' means the booking status is something else.

Your actual result

0 rows. No existing booking had a cancellation record while being marked non-cancelled.

Main takeaway

For paired tables, validate the relationship in both directions.

Lesson 29: Find orphan cancellation records

Concept: Orphan child records

WHAT TO DO	HOW TO DO IT	WHY IT MATTERS
Identify cancellation records whose booking_id does not exist in bookings.	LEFT JOIN cancellations to bookings and keep rows where the booking side is NULL.	A cancellation without its parent booking cannot be fully reconciled or analyzed in normal booking context.

SQL

```
SELECT
    c.cancellation_id,
    c.booking_id
FROM raw.cancellations AS c
LEFT JOIN raw.bookings AS b
    ON c.booking_id = b.booking_id
WHERE b.booking_id IS NULL;
```

How the SQL works

The left table is now cancellations because we want to preserve every cancellation and test whether a parent booking exists.

Your actual result

15 orphan cancellation records. First observed orphan booking_id: B_ORPHAN_011.

Main takeaway

An orphan child record has a foreign-key-like value with no corresponding parent record.

Lesson 30: Find bookings with multiple payment rows

Concept: Table grain and one-to-many relationships

WHAT TO DO	HOW TO DO IT	WHY IT MATTERS
Count how many payment transactions each booking has and show bookings with more than one.	GROUP BY booking_id in raw.payments and use HAVING COUNT(*) > 1.	Multiple payment rows are not necessarily duplicates. Payments have transaction grain, so one booking may have installments, refunds, or multiple attempts.

SQL

```
SELECT
    booking_id,
    COUNT(*) AS payment_rows
FROM raw.payments
GROUP BY booking_id
HAVING COUNT(*) > 1
ORDER BY payment_rows DESC;
```

How the SQL works

The important idea is grain: bookings and payments do not represent the same level of detail.

Your actual result

245 booking IDs had more than one payment row. First observed booking: B0088132 with payment_rows = 2.

Main takeaway
Never call repeated booking_ids in a transaction table duplicates until you understand that table's grain.

Lesson 31: Aggregate payments before joining to booking grain

Concept: Preventing double counting

WHAT TO DO	HOW TO DO IT	WHY IT MATTERS
Create one payment-summary row per booking before joining it to bookings.	Aggregate payments in a CTE, then join the summarized result to the one-row-per-booking table.	Directly joining one booking to several payment rows can repeat booking-level amounts such as GBV and overstate totals.

SQL

```
WITH payment_summary AS (
    SELECT
        booking_id,
        COUNT(*) AS payment_rows
    FROM raw.payments
    GROUP BY booking_id
)
SELECT
    b.booking_id,
    b.gross_booking_value,
    p.payment_rows
FROM raw.bookings AS b
LEFT JOIN payment_summary AS p
    ON b.booking_id = p.booking_id
WHERE b.booking_id = 'B0088132';
```

How the SQL works

The CTE changes the payment data to booking grain: one row per booking.

Only after that do we join to bookings.

Your actual result
B0088132 had gross_booking_value = 9078.85 and payment_rows = 2, while the booking GBV remained one booking-level value.

Main takeaway
Golden rule: when joining tables with different grains, aggregate the many-side first if the analysis requires one row per booking.

Lesson 32: Find Cancelled + Paid bookings

Concept: Business-state consistency and process timing

WHAT TO DO	HOW TO DO IT	WHY IT MATTERS
Identify bookings that are marked Cancelled but whose payment_status is still Paid.	Filter raw.bookings on both conditions.	This combination may indicate a refund is pending, the status has not been updated, or the booking was paid before later cancellation. It requires reconciliation, not automatic rejection.

SQL

```
SELECT
    booking_id,
    booking_status,
    payment_status
FROM raw.bookings
WHERE booking_status = 'Cancelled'
  AND payment_status = 'Paid';
```

How the SQL works

AND means both conditions must be true.

Your actual result

1,000 Cancelled + Paid booking rows. First observed booking_id: B00000116.

Main takeaway

A suspicious combination can be a legitimate business state. Always investigate process context before labeling it an error.

Lesson 33: Inspect refund information for a suspicious booking

Concept: Record-level reconciliation

WHAT TO DO	HOW TO DO IT	WHY IT MATTERS
Look up the cancellation record for one Cancelled + Paid booking and inspect refund-related fields.	Query raw.cancellations by booking_id.	A sample inspection helps you understand whether the suspicious status combination is explained by downstream refund activity.

SQL

```
SELECT *
FROM raw.cancellations
WHERE booking_id = 'B00000116';
```

How the SQL works

This is a focused drill-down, not a full reconciliation yet.

Your actual result

For B00000116, refund_amount = 390 and cancellation_fee = 0.

Main takeaway

Cancelled + Paid does not automatically mean bad data. To determine full vs partial refund, compare refund amount with the original paid amount in a later reconciliation step.

3. Final Phase 1 Data-Quality Summary

The following issue log summarizes the actual findings from the completed Phase 1 profiling exercise. It separates confirmed data-quality issues from observations that require business investigation.

Check	Observed result	Assessment	Recommended Phase 2 treatment
Total raw booking rows	120,175	Baseline	Use as reconciliation baseline
Unique booking IDs	120,000	Baseline	Expected booking-level distinct count
Duplicate booking rows	175	High	Deduplicate later in staging/clean; preserve raw
Missing supplier_id	240	Medium	Keep booking with missing-supplier flag; handle carefully in supplier analysis
Mixed booking-date formats	161 DD/MM/YYYY vs 120,014 YYYY-MM-DD	Medium	Parse and standardize to DATE in staging
Travel date before booking date	25	High	Flag/exclude from lead-time-sensitive clean analysis
rooms <= 0	20	High	Flag as invalid business-rule records
nights <= 0	20	High	Flag as invalid business-rule records
GBV <= 0	58	High	Investigate; exclude from normal financial KPI logic if invalid
supplier_cost > GBV	58	Review	Possible real loss or bad data; investigate context
discount_amount > GBV	39	Review	Investigate adjustment/pricing logic
Booking-status categories	3 clean categories	Good	No category standardization needed for status
Currencies	15	Important	Normalize to one reporting currency before financial totals
Country-name consistency	Multiple variants	Medium	Standardize in staging with TRIM/CASE rules
Missing agent matches	1,000 booking rows	High	Flag or exclude from agent-dimension analysis
Missing hotel matches	0	Good	No action
Invalid nonblank supplier references	0	Good	No action; separate from 240 blanks
Missing destination matches	0	Good	No action
Cancelled without cancellation row	0	Good	No action
Cancellation linked to non-cancelled booking	0	Good	No action
Orphan cancellation records	15	High	Log as exceptions; cannot reconcile to parent booking
Bookings with multiple payment rows	245 booking IDs	Expected relationship	Aggregate payment table to booking grain before joining
Cancelled + Paid bookings	1,000	Review	Reconcile against refund/payment transactions

4. Phase 1 Decision Framework

After profiling, each issue should be assigned a treatment rather than handled impulsively. A simple four-part framework is useful:

Treatment	Meaning	Example from this project
Correct	Standardize a value when the intended value is known with reasonable confidence.	Country variants such as india / INDIA -> India; known Signapore -> Singapore.
Standardize	Convert valid values into one consistent representation.	Text dates -> DATE; country case; reporting currency conversion later.
Exclude / Flag	Keep raw data but omit invalid records from a specific clean analysis or create a DQ flag.	Negative nights, zero rooms, travel before booking.
Escalate / Investigate	Do not decide from SQL alone; business context is required.	Supplier cost > GBV, discount > GBV, Cancelled + Paid.

Important principle
Raw data should remain unchanged. Phase 2 will create staging and clean layers where conversion, standardization, deduplication, flags, and exception handling are applied reproducibly.

5. SQL Concepts Learned in Phase 1

Concept	Why it matters for a Data Analyst
SELECT / FROM / LIMIT	Read a controlled sample of a table.
COUNT(*)	Count rows.
DISTINCT	Remove repeated values for uniqueness checks.
GROUP BY	Aggregate rows by category or key.
HAVING	Filter aggregated groups.
WHERE	Filter individual rows.
FILTER	Conditional aggregate counts in PostgreSQL.
CASE	Create rule-based classifications or transformations.
TRIM / UPPER / INITCAP	Normalize text for comparison and display.
Regular expressions (~)	Identify date/text patterns.
::date / ::numeric	Cast text to a typed value.
TO_DATE	Parse a known date-text format.
CTE / WITH	Build readable multi-step SQL.
Calculated columns / AS	Create diagnostic or KPI-like derived values.
LEFT JOIN	Keep all rows from one table while attempting to match another.
IS NULL after LEFT JOIN	Find orphan or unmatched foreign-key values.
Table grain	Understand what one row represents before joining or aggregating.

Concept	Why it matters for a Data Analyst
One-to-many joins	Recognize how repeated child rows can duplicate parent-level measures.

6. Common Mistakes to Avoid

- Deleting suspicious raw rows immediately. First profile, investigate, and preserve raw data for traceability.
- Assuming every repeated ID is a duplicate. In payments, repeated booking_id values can be valid because the table grain is one transaction per row.
- Summing monetary values across different currencies before FX normalization.
- Joining bookings directly to payments and then summing booking-level GBV, which can double count bookings with multiple payment rows.
- Treating a missing value and an invalid foreign-key reference as the same issue.
- Assuming supplier_cost > GBV or Cancelled + Paid is automatically erroneous without checking business process context.
- Casting messy text directly to numeric/date without first understanding possible formats or invalid strings.
- Reporting only the presence of an issue without quantifying how many rows are affected.

7. Reusable Phase 1 SQL Checklist

The following condensed checklist can be reused on future projects. It is not a replacement for understanding the business; it is a practical sequence for first-pass profiling.

1. Inspect sample rows and confirm table grain.
2. Count total rows.
3. Compare total rows to distinct primary-key-like IDs.
4. List duplicate keys and inspect sample duplicate records.
5. Profile NULL and blank values for important columns.
6. Classify and parse date formats safely.
7. Check logical date sequences and calculate useful date differences.
8. Validate numeric ranges and cross-field business rules.
9. Profile categorical distributions and text consistency.
10. Validate foreign-key relationships with LEFT JOIN + IS NULL.
11. Check consistency across operational tables such as bookings and cancellations.
12. Understand one-to-many table grain before joining payments or events.
13. Document every issue with count, severity, business meaning, and intended treatment.

8. Consolidated SQL Reference

These are the core profiling patterns from Phase 1. Keep them as a study reference and adapt table/column names for new projects.

Row count

```
SELECT COUNT(*) FROM raw.bookings;
```

Unique IDs and duplicate count

```
SELECT COUNT(*) AS total_rows, COUNT(DISTINCT booking_id) AS unique_booking_ids, COUNT(*) -
COUNT(DISTINCT booking_id) AS duplicate_rows FROM raw.bookings;
```

Duplicate IDs

```
SELECT booking_id, COUNT(*) AS row_count FROM raw.bookings GROUP BY booking_id HAVING COUNT(*) > 1
ORDER BY row_count DESC, booking_id;
```

Missing field example

```
SELECT COUNT(*) FROM raw.bookings WHERE supplier_id IS NULL OR TRIM(supplier_id) = '';
```

Conditional count

```
SELECT COUNT(*) FILTER (WHERE booking_status = 'Cancelled') AS cancelled_bookings FROM raw.bookings;
```

Invalid date order

```
WITH p AS (SELECT booking_id, CASE WHEN booking_date ~ '^\d{4}-\d{2}-\d{2}$' THEN
booking_date::date ELSE TO_DATE(booking_date,'DD/MM/YYYY') END AS booking_dt, CASE WHEN travel_date
~ '^\d{4}-\d{2}-\d{2}$' THEN travel_date::date ELSE TO_DATE(travel_date,'DD/MM/YYYY') END AS
travel_dt FROM raw.bookings) SELECT * FROM p WHERE travel_dt < booking_dt;
```

Numeric rule

```
SELECT booking_id, nights FROM raw.bookings WHERE nights::numeric <= 0;
```

Category distribution

```
SELECT booking_status, COUNT(*) AS row_count FROM raw.bookings GROUP BY booking_status ORDER BY
row_count DESC;
```

Missing foreign-key match

```
SELECT b.booking_id, b.agent_id FROM raw.bookings b LEFT JOIN raw.travel_agents a ON b.agent_id =
a.agent_id WHERE a.agent_id IS NULL;
```

Orphan cancellation

```
SELECT c.cancellation_id, c.booking_id FROM raw.cancellations c LEFT JOIN raw.bookings b ON
c.booking_id = b.booking_id WHERE b.booking_id IS NULL;
```

Multiple payment rows

```
SELECT booking_id, COUNT(*) AS payment_rows FROM raw.payments GROUP BY booking_id HAVING COUNT(*) >
1 ORDER BY payment_rows DESC;
```

Cancelled + Paid

```
SELECT booking_id, booking_status, payment_status FROM raw.bookings WHERE booking_status =
'Cancelled' AND payment_status = 'Paid';
```

9. What Phase 1 Prepares You for in Phase 2

Phase 1 answered "What is wrong, inconsistent, risky, or important in the raw data?" Phase 2 will answer "How do we create trusted analysis-ready data without destroying the raw evidence?"

- Create staging and clean schemas.
- Convert raw text to correct data types safely.
- Standardize text categories.
- Deduplicate bookings reproducibly.
- Create data-quality flags and exception tables.
- Handle missing and invalid foreign keys intentionally.
- Build a clean booking fact table.
- Aggregate payments to booking grain to prevent double counting.
- Validate that the clean layer reconciles back to the raw source.

You are ready for Phase 2 when...

You can explain each Phase 1 issue in plain language, reproduce the SQL check, and describe why the issue matters to a business KPI or join.

END OF PHASE 1 DOCUMENTATION

Next: Phase 2 - Staging and Clean Data Layers